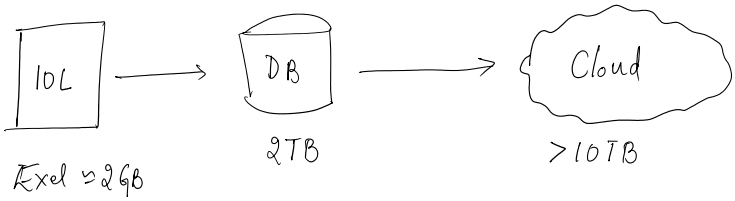
Big Data ⇒



Large & Complex data which is too big for traditional methods to Handle / process

⇒ 5v's ⇒ Volume, Velocity, Variety, Veracity (Quality), Value

⇒ Storage & HDFS ⇒ Storage infra & Distributed file system

⇒ Map-reduce ⇒ Model flow & limitations

⇒ Spark & RDD's ⇒ Architecture & RDD's

⇒ Comparison of Map-reduce & Spark

⇒ Case study ⇒ Implementation of Above

Distributed file system ⇒ Data is not stored in a single place, Ex HDFS

↳ Data will be spread across different clusters / machine to handle volume & parallel processing

Components ⇒

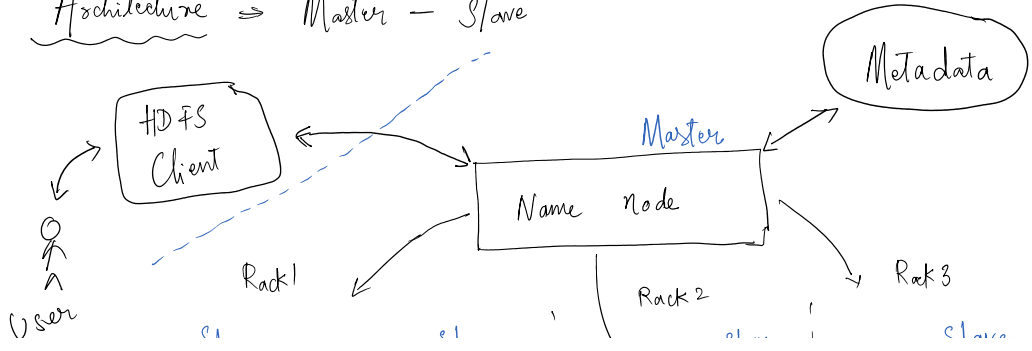
① Distribution of files ⇒ Break large files & store in small machines ⇒ Replication ((fault tolerace))

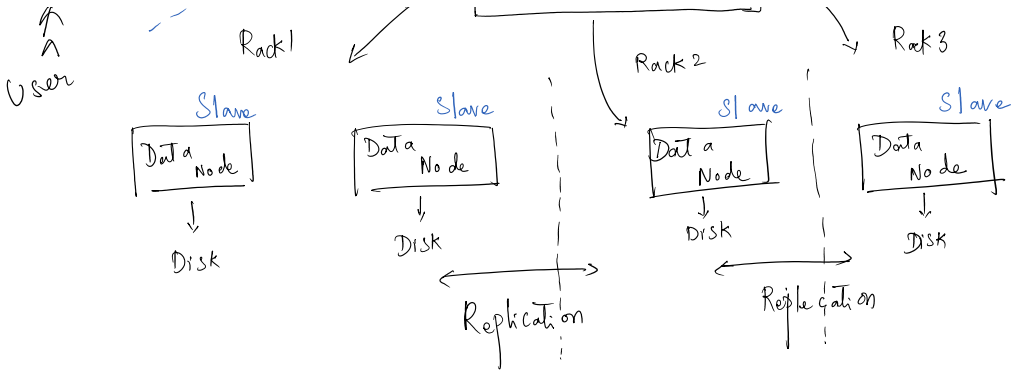
↳ If one machine fails ⇒ Data can be recovered from other machine ↑

② No-SQL DB ⇒ Flexible, Semi / Unstructured, Doc, tables, (Key-Value) etc ⇒ Metadata

③ Objects ⇒ Unstructured data "object" ⇒ identifier "metadata" ⇒ Hierarcial flat file

Architecture ⇒ Master – Slave





Map reduce ⇒ Break the task → Process in parallel

① Input ⇒ Data(I/p) ⇒ broken in to chunks

② ⇒ (Key : Value) ("apple" : 1) ⇒ All mappers run in parallel

[I am apple → 26 ; I ate Apple → 84] 6

⇒ (Metadata)

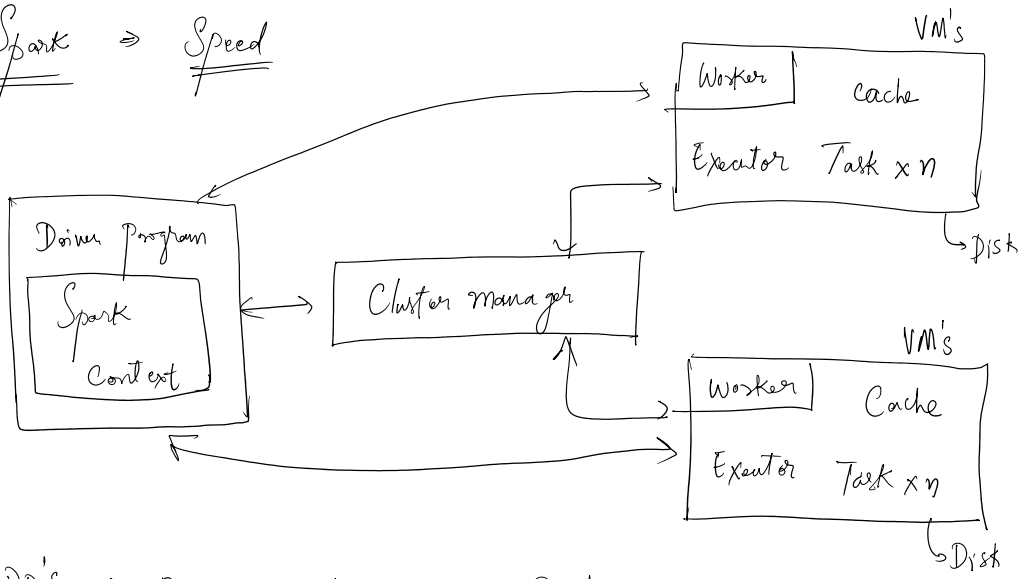
[(I x2) (am x1) (ate x1) (apple x2)] 4

③ Shuffle & sort ⇒ group values with same key → sort them

④ Reduce ⇒ apple x3 → ("apple" : 3)

⑤ O/p ⇒ Reduced Data

Spark ⇒ Speed



DAG → Directed Acyclic graph

RDD's ⇒ Resilient, Distributed, Immutable, Lazy execution